LABORATORY MANUAL · WEEK 6

Lab 06 — Your First Continuous Integration Pipeline

DevOps Principles and Practices (2+1) · BS Computer Science · National Textile University

Duration	3 hours (one laboratory session)
Mapped outcome	L-CLO-1 — Construct an automated continuous integration pipeline using industry-standard tools. (P3 — Precision, Psychomotor). This is the lab the outcome is named after.
Graduate attribute	GA-5 — Modern Tool Usage; GA-3 for the trigger and gate decisions
Prerequisite	Lab 05 complete and on your team repository's `main`: `package.json`, `package-lock.json`, a `Makefile` with a working `ci` target, and `tools/ src/ test/`
Environment	Ubuntu 24.04 under WSL2 (or native Linux / macOS Terminal), a browser, and a GitHub account
Deliverable	Run URLs, one blocked pull request with a screenshot, and a short report — see Section 9
Weight	Counts towards the 35 % weekly laboratory task component

> **Everything today happens in your Lab 04 team repository.** Not your personal `devops-lab`, and not your fork of the class repository. The team repository is the one with branch protection on `main`, and today that protection starts doing real work. If your team never created one, tell the instructor in the first five minutes.

1. Why this lab is the one the outcome is named after

You have spent five labs building one command. Lab 01 made `build.sh`. Lab 02 turned it into a Makefile with a `ci` target. Lab 05 made that target install dependencies from a lock file, build, test and package a versioned artifact.

Today somebody else's computer runs it. Not as a convenience — as an authority. By the end of the session your team's `main` branch will refuse a pull request whose build is red, and no amount of being in a hurry will change its mind.

Five things happen today:

1. You prove your build works from a **clean clone**. Many of you will fail this, and it is better to fail it here than on the agent.
2. You write a workflow, push it, and watch a machine you have never logged into run your build.
3. You **break it three times on purpose** and learn to read three different kinds of failure — what the outline calls "interpreting build results".
4. You make the build result **required**, and prove that a red pull request cannot merge.
5. You collect the artifact the server built, turn on caching, and decide which jobs deserve which triggers.

> **A note on what is being assessed.** Task 4 is the marked one — the gate, and the evidence that it works. Tasks 1 to 3 build up to it and Task 5 tunes it. Your evidence is URLs: a green run, a red run, and a pull request that could not be merged.

The outline offers a choice: install a build server such as Jenkins in a container, **or** configure GitHub Actions. This manual uses GitHub Actions, because your code is already on GitHub from Lab 04, and because a three-hour session spent installing Jenkins teaches you about Jenkins rather than about continuous integration. Section 12 maps everything here onto Jenkins.

2. Objectives

By the end of this session you will be able to:

- Explain why a build that passes on your laptop proves nothing, and prove your own build from a clean clone.
- Write a workflow that triggers on a push and on a pull request, and say what every line is for.
- Read a build log and tell a failing test, a dependency problem and a missing file apart from their output alone.
- Make a build result a **required status check**, and demonstrate that it blocks a merge.
- Explain, with an example you produced yourself, what "require branches to be up to date" prevents.

- Collect a build artifact produced by the server rather than by a developer.
- Choose which jobs run on which triggers, and justify the choice from the agent budget and the suite length.

3. Before you start

3.1 Check your team repository is where Lab 05 left it

```
cd ~/devops/devops-project-TEAMNAME
git switch main && git pull
git log --oneline -3
ls
```

You should see `Makefile`, `package.json`, `package-lock.json`, `tools/`, `src/` and `test/` on `main`. If the build tool never reached `main` in Lab 05, do that first — Task 1 depends on it.

> **If your team has no working Lab 05 project.** The provided `fallback/` folder is the Lab 05 model answer: Makefile, package.json, tools, src and test. Run `npm install` once to create the lock file, commit all of it to your team repository through a pull request, and carry on — and **say so in your report**. You lose no marks for using it. You lose marks for not saying.

3.2 Check you still have your Lab 04 ruleset

On GitHub: your team repository → **Settings → Rules → Rulesets**. You should see the ruleset you created in Lab 04, with **Enforcement status: Active**. Task 4 edits it. If it says Disabled, fix that now; a ruleset on Disabled does nothing, and you will spend twenty minutes wondering why.

3.3 Know where the Actions tab is

Your repository → **Actions**. It is empty right now. By the end of Task 2 it will not be.

1 Make the build work from a clean clone — 25 minutes

Not assessed directly. It is the most valuable twenty-five minutes in this lab, because every failure in Tasks 2 and 3 is really this failure arriving later and more expensively.

4.1 The claim you are about to test

Your build works. You know this because you have run it. But you have run it in a directory that has been accumulating state since Lab 05 — installed packages, files you created and never committed, an `out/` directory from an earlier run. The agent will have none of that.

So test the real claim: that **the repository** builds, not that your directory builds.

```
cd ~/devops
git clone https://github.com/OWNER/devops-project-TEAMNAME.git fresh-check
cd fresh-check
ls -a
make ci
```

> **Clone into a NEW directory with a different name.** The point is a copy that contains exactly what is committed and nothing else. Pulling into your existing directory tests nothing, because everything missing from the repository is still sitting on your disk.

4.2 What a pass looks like

Expected output

```
npm ci

added 7 packages, and audited 8 packages in 694ms
...
check: 3 source file(s) in src
mkdir -p out
  BUILD src/greeting.md -> out/greeting.html
  BUILD src/notes.md -> out/notes.html
  BUILD src/services.md -> out/services.html
...
npm pack
...
devops-pipeline-1.1.0.tgz
```

Note that `npm ci` ran. In your own working directory it usually does not, because `node_modules` is already there and newer than the lock file. In a fresh clone there is no `node_modules` at all, so the dependency install is part of every build — which is why the lock file matters, and why Task 5 caches it.

4.3 The ways this fails, and what each means

What you see	What it means, and the fix
`No rule to make target`, or `no such file or directory`	A file your build needs was never committed. You created it in Lab 05 and it has sat untracked ever since. `git status` in your **old** directory shows it. Commit it.
`npm ci can only install packages when your package.json and package-lock.json ... are in sync`	You committed `package.json` without the regenerated lock file. Run `npm install` once in your old directory, then commit **both** files together. Always both.
It builds, but produces the wrong number of pages	A source file exists on your laptop and not in the repository. The clone is not wrong; your repository is incomplete.
`node_modules` appears in the clone	It was committed by mistake. `git rm -r --cached node_modules`, confirm `node_modules/` is in `.gitignore`, and commit.

Fix whatever you find, push, and clone again into a second new directory. Repeat until a brand-new clone builds first time. Then delete the check-out directories; they have done their job.

4.4 Checkpoint

- A clone made in the last five minutes runs `make ci` green with no manual steps.
- `git status` in your working directory is clean, or shows only files you have deliberately left out.
- You can say why `npm ci` ran in the clone and not in your own directory.

2 Your first pipeline run — 30 minutes

Not assessed directly.

5.1 Write the workflow

Workflows live in a fixed place. The directory name and the extension both matter; GitHub will not look anywhere else.

```
cd ~/devops/devops-project-TEAMNAME
git switch -c add-ci
mkdir -p .github/workflows
```

Create `.github/workflows/ci.yml` containing exactly this:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    name: build-and-test
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '22'

      - name: Build, test and package
        run: make ci
```

> **YAML is indentation-sensitive and will not forgive you.** Two spaces per level, spaces only — **never a tab**. A tab produces an error that does not mention tabs. If GitHub says your workflow is invalid, suspect the indentation before anything else.

5.2 Read it before you push it

Line	What it does
`push: branches: [main]`	Run when a commit lands on `main`. This is the build that produces the artifact you would deploy.
`pull_request:`	Run when a pull request is opened or updated. **This is the one that becomes the gate in Task 4.**
`name: build-and-test`	The job's name. It is also the name of the status check GitHub reports — Task 4 requires it by this name.
`runs-on: ubuntu-24.04`	Which agent. A clean Ubuntu machine, created for this run and destroyed after it.
`actions/checkout@v4`	Clone the repository onto the agent. Without it the agent has an empty directory.
`actions/setup-node@v4`	Install Node 22. The agent does not have your Node, or any particular Node.
`run: make ci`	Your target, unchanged since Lab 02. The server has no idea what is inside it, and does not need to.

That last row is the point of the whole file. Everything you built over five weeks sits behind one line — which is why you can test your build without pushing, and why moving to a different CI tool later is a ten-line change.

5.3 Push it and watch

```
git add .github/workflows/ci.yml
git commit -m "Add CI workflow"
```

```
git push -u origin add-ci
```

Open a pull request. Then go to the **Actions** tab: a run is already in progress, triggered by `pull_request`. Open it, click **build-and-test**, and expand each step.

5.4 Read the log properly

Do not just look for the green tick. Find these four things and write them in your report:

1. The **total duration** of the run.
2. The line where `npm ci` reports how many packages it added.
3. The three `BUILD src/... -> out/...` lines, proving the agent built the same pages you build.
4. The `Set up job` step at the top, which names the runner image and its version. You have never seen this machine, and in an hour it will not exist.

> **The agent started with nothing.** No `node_modules` you warmed, no file you forgot to commit, no environment variable you set months ago. That is why its verdict is worth something — and why Task 1 mattered. If Task 1 passed, this run is green. If you skipped Task 1, this run is where you find out.

5.5 Merge it

Get a teammate to approve, and merge. A second run starts at once, triggered by the `push` to `main`. Its duration is about the same: the agent gets no credit for work the previous run did.

5.6 Checkpoint

- A green run exists, triggered by a pull request.
- A second green run exists, triggered by the merge to `main`.
- You have both durations, and can say why they are about the same.

3 Break it three times, and read the result — 30 minutes

Not assessed directly — but two rubric lines depend on it, and it is the part of this session you will actually use at work.

A green build teaches you nothing. Work on a throwaway branch so none of this reaches `main`, and open a pull request from it so that every push runs the pipeline:

```
git switch main && git pull
git switch -c break-it
```

For each failure below: make the change, push, open the run, and **before you read the error, predict which step will be red**.

6.1 Failure one — a failing test

Append a test that is simply not true to `test/build.test.js`:

```
test('every page names the course', () => {
  for (const p of pages) {
    assert.match(fs.readFileSync('out/' + p, 'utf8'), /DevOps/);
  }
});
```

```
git commit -am "Add a test that fails"
git push -u origin break-it
```

Expected output

```
✖ every page names the course
...
ℹ fail 1
...
AssertionError [ERR_ASSERTION]: The input did not match the regular expression /DevOps/
...
make: *** [Makefile:NN: test] Error 1
Error: Process completed with exit code 2.
```

Three things to notice. The **build step** is red but the steps before it are green, so the failure is located. Make reports `Error 1` (the `NN` is the line in your Makefile) and the agent reports a non-zero exit code — that is how every build tool tells every build server that something failed. And **no artifact was produced**, because `package` depends on `test` and Make refused to run it.

> **This is the failure you want.** It is specific, it names the assertion, and the fix is obvious. Remove the test, push, and watch it go green. The other two failures are the ones that waste afternoons.

6.2 Failure two — a dependency the lock file has not approved

Add a dependency to `package.json` **by hand**, without running `npm install`:

```
"dependencies": {
  "markdown-it": "^15.0.3",
  "js-yaml": "^4.1.0"
}
```

```
git commit -am "Add js-yaml"
git push
```

Expected output

```
npm error code EUSAGE
npm error
npm error `npm ci` can only install packages when your package.json and
npm error package-lock.json or npm-shrinkwrap.json are in sync. ...
npm error
npm error Missing: js-yaml@4.3.2 from lock file
npm error Missing: argparse@2.0.1 from lock file
```

Now the failure is at **install**, and the build never started. Note that npm names two missing packages when you added one: `argparse` is `js-yaml`'s own dependency, and the lock file has to approve that too. Your version numbers may differ; the shape will not.

The fix is the Lab 05 rule, and it is worth saying aloud: remove `js-yaml` again (you do not need it), or run `npm install` and commit `package.json` and `package-lock.json` **in the same commit**. Push, and watch it go green.

6.3 Failure three — the one that will actually catch you

Create a new source file, and commit **everything except that file**:

```
printf '# Pricing\n\nSee the rate card.\n' > src/pricing.md
make ci                          # four pages, green, on your laptop
ls out/
git commit --allow-empty -m "Add the pricing page"
git push
```

On your laptop the build produced **four** pages. Open the run on the agent: it produced **three**, and it is **green**. `src/pricing.md` was never committed, so the clone has never heard of it, and nothing in the test suite knows a fourth page was expected.

> **A green build that built the wrong thing. This is "works on my machine".** There is no error message that says "you forgot to commit a file", because from the repository's point of view nothing is missing. The agent did exactly what the repository told it to. The only defences are running `git status` before every push, and Task 1: asking whether a clean clone builds what you think it builds.

Commit the file, push, and confirm the agent now builds four pages. Then throw the branch away:

```
git add src/pricing.md && git commit -m "Actually add the pricing page"
git push
# once you have seen four BUILD lines in the run:
git switch main
git branch -D break-it
git push origin --delete break-it
```

6.4 Checkpoint

- Two red runs exist, red for different reasons, and one green run that built three pages instead of four.
- You have the URL of each, and can say what happened in each from the log alone.
- You can state the difference between a build that fails and a build that succeeds at the wrong thing.

4 ASSESSED — make the build result a gate 45 minutes

This task is marked. Section 10 is the rubric. The marks are for the gate actually working. Your evidence is a pull request with a failed required check and a merge button that will not let you press it.

7.1 Why a status check is not yet a gate

Your workflow already reports pass or fail against every pull request. Right now that is **information**: the tick is there, and nothing stops anyone merging past a cross. In Lab 04 you deliberately left "Require status checks to pass" alone, because there were no checks to require. Now there is one.

7.2 Require the check

1. Settings → Rules → Rulesets → open your `protect-main` ruleset.
2. Confirm **Enforcement status** is **Active**.
3. Tick **Require status checks to pass**.
4. Click **Add checks**, type `build-and-test`, and select it. That is the `name:` of your **job** — which is why the workflow gave the job a name.
5. Leave **Require branches to be up to date before merging** unticked **for now**. You turn it on in 7.5.
6. Save.

If the check does not appear when you search for it. GitHub only suggests checks it has seen report recently. Push to any branch with an open pull request, let the run finish, and search again. Do not type a name by hand that does not exactly match the job name: a required check that never reports can never pass, and nothing will ever merge again.

7.3 Prove a red pull request cannot merge

This is the evidence for the rubric. Make a change that is certainly red:

```
git switch main && git pull
git switch -c prove-the-gate
printf "\ntest('this always fails', () => assert.ok(false));\n" >> test/build.test.js
git commit -am "Deliberately break the build to test the gate"
git push -u origin prove-the-gate
```

Open a pull request and wait for the run to finish. At the bottom of the pull request page you should see the `build-and-test` check failed and marked **Required**, and a merge button that is disabled.

Capture this. It is the marked artefact. Screenshot the pull request showing the failed required check and the disabled merge button, and record its URL. A description is not evidence; the URL and the screenshot are.

Now fix it on the same branch and watch the gate open:

```
git checkout HEAD~1 -- test/build.test.js
git commit -m "Remove the deliberate failure"
git push
```

The pull request re-runs itself, the check goes green, and the merge button becomes available. **Do not merge it.** Close it instead — it has done its job, and the report asks for its URL.

7.4 The gap the gate still has

Your gate now guarantees that every branch was green **at some point**, against whatever `main` looked like when that branch was built. That is not the same as guaranteeing `main` stays green, and the difference is the Week 03 semantic conflict.

Produce one yourself, with a partner. One of you takes branch A, the other branch B, **both cut from the same main**:

Branch	The change
A (`feature-a`)	In `tools/build.js`, make the page title prettier — hyphens become spaces and each word is capitalised — by replacing the `const title = ...` line with the snippet below.
B (`feature-b`)	In `test/build.test.js`, add a test asserting that each page's `<title>` is exactly its source filename — the snippet below.

Branch A, in `tools/build.js`:

```
const title = path.basename(src, '.md')
  .replace(/-/g, ' ')
  .replace(/\b\w/g, (c) => c.toUpperCase());
```

Branch B, appended to `test/build.test.js`:

```
test('each page title is its source filename', () => {
  for (const p of pages) {
    const html = fs.readFileSync('out/' + p, 'utf8');
    assert.ok(html.includes('<title>' + p.replace('.html', '') + '</title>'));
  }
});
```

Each of you pushes and opens a pull request. **Both are green** — check that they are, because that is the whole point. Merge A. Now look at B: it is **still showing green**, because its check ran before A landed and nothing has re-run it.

Merge B. Then open the `push` run on `main`, which builds the two together for the first time:

Expected output

```
✖ each page title is its source filename
...
ℹ fail 1
```

> **Neither of you did anything wrong, and git reported no conflict.** You changed different files. Both branches were green. Both reviews passed. `main` is broken, by a combination that no machine had ever executed before it landed. This is a semantic conflict, and it is the single best argument for the setting in 7.5.

Fix `main` by the Week 06 rule — revert first, discuss afterwards. Open B's merged pull request and click **Revert**. GitHub opens a new pull request that undoes it; merge that once it is green. (Your ruleset forbids pushing to `main` directly, so the revert goes through a pull request too.) Note how long `main` was red — Week 06 told you what that costs.

7.5 Close the gap

1. Settings → Rules → Rulesets → your ruleset → under **Require status checks to pass**, tick **Require branches to be up to date before merging**.
2. Save.

Now prove it, with a fresh pair of changes — again one each, both cut from the current `main`:

Branch	The change
C (`feature-c`)	Rename a source file: `git mv src/services.md src/our-services.md`
D (`feature-d`)	Append a test that the services page exists — the snippet below.

```
test('the services page exists', () => {
  assert.ok(fs.existsSync('out/services.html'));
});
```

Both pull requests go green on their own. Merge C. Now look at D: GitHub reports that **the branch is out of date with the base branch**, and the merge button stays disabled until it is updated. Click **Update branch**.

D re-runs against a `main` that now contains C — and goes **red**, before it could do any damage. That is the setting working: every branch must be green against the `main` it is actually merging into, not the one it started from. Close D without merging; it was the experiment.

> **This is the trade, and you should be able to state it.** The cost: every branch must update and re-run whenever `main` moves, which on a busy repository is a lot of waiting. The benefit: a class of failure that no test on a single branch can catch becomes impossible. Small team, slow-moving `main` — switch it on. Large team, `main` moving every few minutes — it becomes a queue, and teams use a merge queue instead.

7.6 Checkpoint — this is what is marked

- `build-and-test` is a required status check on `main`, and the ruleset is Active.
- A pull request exists whose merge was blocked by a failed required check, with its URL and a screenshot.
- You produced a semantic conflict: two green branches that broke `main` together.
- "Require branches to be up to date" is on, and you have seen it force an update and catch the conflict.
- You can state what that setting costs as well as what it buys.

5 The artifact, the cache, and the trigger set — 30 minutes

Compulsory, and it carries rubric marks. Make these changes on a branch and merge them through a pull request — your gate now applies to you.

8.1 Collect the artifact the server built

Right now `make ci` produces a `.tgz` on the agent, and then the agent is destroyed with it. Keep it. Add a final step to the `build` job:

```
      - uses: actions/upload-artifact@v4
        with:
          name: devops-pipeline
          path: '*.tgz'
          if-no-files-found: error
```

Push, let the run finish, and look at the bottom of the run's summary page: there is now an **Artifacts** section containing `devops-pipeline`. Download it. GitHub wraps it in a zip; unpack that, then look inside the tarball:

```
unzip devops-pipeline.zip
tar -tzf devops-pipeline-*.tgz
```

Expected output

```
package/tools/build.js
package/README.md
package/package.json
```

That file was built by a machine nobody logged into. In Lab 05 you made the same artifact on your laptop. This one came from a clean clone, installed from the lock file, with the tests passing. It is the first artifact in this course that is genuinely build-once-deploy-many — and `if-no-files-found: error` makes the run fail rather than silently upload nothing.

8.2 Make it faster

Note how long the install step takes. Now cache it — one line, in the `setup-node` step you already have:

```
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: npm
```

Push twice. The first run populates the cache; the second restores it. Record both durations, and find the cache step in the second run's log.

Notice what the cache is keyed on. `setup-node` keys the npm cache on your `package-lock.json`. Change a dependency and the key changes, so the cache is correctly missed. This works only because you have a lock file — Week 05's point, arriving as a speed-up.

Never cache the build output. Caching the dependency download is fine. Caching `out/` or the `.tgz` is not: the whole value of the agent is that it starts empty and rebuilds from source. An agent that reuses yesterday's output cannot tell you whether a clean build still works.

On a project this small the saving is seconds, and may be lost in the noise. That is a finding, not a failure — write down what you measured. On a real project the install is usually the largest single serial block, which is why Week 06 said to cache it before buying agents.

8.3 Decide the trigger set

Add a slow job — a dependency audit — and decide where it belongs. Append this under `jobs:`, at the same indentation as `build:`:

```
  audit:
    name: dependency-audit
    runs-on: ubuntu-24.04
    if: github.event_name == 'schedule'
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: npm
      - run: npm ci
      - run: npm audit --audit-level=high
```

And add the trigger that fires it, next to the two you already have:

```
on:
  push:
    branches: [main]
  pull_request:
  schedule:
    - cron: '0 2 * * *'         # 02:00 UTC, nightly
```

The `if:` line is what keeps the audit off every pull request. Without it, the job would run on all three triggers and every push would pay for it.

Job	Trigger, and why
`build-and-test`	Pull request and push. Fast, and it is the verdict that gates the merge.
`dependency-audit`	Schedule only. The answer rarely changes between two Tuesdays, and gating on it would put a slow job in front of every merge.

> **A scheduled workflow only runs from the default branch.** Your `schedule:` does nothing until the file is merged to `main`, GitHub may start it some minutes after the stated time, and it pauses schedules on repositories with no activity for sixty days. Do not spend lab time waiting for 02:00 — note the behaviour and move on.

8.4 Checkpoint

- An artifact is downloadable from a run, and you have unpacked it and checked its contents.
- You have two run durations, before and after caching.
- The audit job exists and is restricted to the schedule.
- You can justify each job's trigger by naming the constraint that decided it.

9. Submission

Submit a single file, `Lab06_<your-roll-number>.md`, plus the one screenshot from 7.3. Everything else lives on GitHub, so this is mostly URLs — each with the sentence that shows you read it.

9.1 The URLs

1. Your team repository, with your final `.github/workflows/ci.yml` pasted in full.
2. A **green** run, with its duration.
3. Your **two red runs** from Task 3, each with one sentence: which step, and why.
4. The green-but-wrong run from 6.3, and how many pages it built.
5. The pull request from 7.3 whose merge was blocked, **plus the screenshot**.
6. The run on `main` that went red after the semantic conflict in 7.4.

9.2 The short answers

- Why does `npm ci` run on the agent but usually not in your own working directory?
- Of the three failures in Task 3, which produced no error at all, and what is the only reliable defence against it?
- What is the difference between a status check and a **required** status check?
- Describe your semantic conflict in three sentences: what A did, what B did, and why neither review could have caught it.
- What does "require branches to be up to date" cost, and when would you turn it off?
- Your run durations before and after caching. What was cached, and what must never be?
- Your trigger table from 8.3, with the constraint that decided each row.
- If you used the `fallback/` Lab 05 project, say so.

> **Open every URL in a private browsing window before you submit.** If a run or a pull request 404s there, your repository is private and the marker cannot see it. Make it public, or add the instructor as a collaborator and say so in the report.

10. Marking rubric

Task 4 carries the marks. The other tasks are assessed through the URLs and the short answers.

Criterion	Weight	What full marks looks like
Clean-clone discipline	10 %	A fresh clone builds with no manual steps; can say why `npm ci` behaves differently there.
A working pipeline	15 %	Triggers on push and pull request, runs `make ci`, is green. Can explain every line of the workflow.
Reading build results	20 %	Two red runs correctly diagnosed from the log; the green-but-wrong run identified as the dangerous one, with the defence stated.
The gate	30 %	A required status check that demonstrably blocks a merge, evidenced by URL and screenshot. Distinguishes a status check from a required one.
The semantic conflict	15 %	Two green branches that broke `main` together; explains why no review or single-branch test could catch it; states what "up to date" costs as well as buys.
Artifact, cache and triggers	10 %	Artifact downloaded and verified; durations recorded honestly; each job's trigger justified by a named constraint.

Three things that cost marks every year. Describing the blocked merge instead of submitting the screenshot and the URL. Calling the semantic conflict "a merge conflict" — it is the opposite, git reported nothing at all. And justifying a trigger with "it is best practice" rather than naming the constraint, which is exactly what the Week 06 lecture said would not earn the mark.

11. Troubleshooting

Symptom	Cause and fix
Nothing appears in the Actions tab	Wrong place or wrong extension. It must be `.github/workflows/ci.yml` — plural `workflows`, leading dot on `.github`.
`Invalid workflow file`	YAML indentation. Two spaces per level, never a tab. The reported line is often a line or two after the real mistake.
`npm ci` fails with `EUSAGE ... in sync`	You committed `package.json` without the lock file. `npm install`, then commit both together.
`npm ci` fails with `The npm ci command can only install with an existing package-lock.json`	The lock file was never committed. Commit it — Lab 05 said always.
Green locally, red on the agent	Something is not committed. Clone into a new directory and run `make ci` there — Task 1 — rather than guessing.
Red locally, green on the agent	Stale state on your laptop. `make clean`, delete `node_modules`, `npm ci`, try again.
The required check is stuck on "Expected — Waiting for status"	The name in the ruleset does not match the job `name:` exactly, so the check never reports. Fix the ruleset.
Merge disabled, but every check is green	"Require branches to be up to date" is on and `main` has moved. Click **Update branch**, let it re-run, then merge.
`Resource not accessible by integration`	A permissions problem. Nothing in this lab needs write permissions — check you have not pasted a step from elsewhere.
The scheduled job never runs	Schedules run only from the default branch, start once the file is on `main`, can be minutes late, and pause after 60 days of inactivity.
Artifact upload: no files found	The build never reached `npm pack`, or something removed the `.tgz` afterwards. The `path:` pattern is relative to the repository root.
Every run sits in the queue	The whole class is sharing free runners. That is the agent-budget constraint from the lecture, live.

12. What a Jenkins lab would have looked like

The outline allows either path, and the concepts are identical — only the vocabulary and the operational burden differ. If your employer uses Jenkins, this is the mapping:

Here	In Jenkins
`.github/workflows/ci.yml`	`Jenkinsfile` in the repository root
`jobs:` then `steps:`	`pipeline { stages { stage { steps { ... } } } }`
`runs-on: ubuntu-24.04`	`agent { docker { image 'node:22' } }`
A GitHub-hosted runner	A node you install, patch and pay for yourself
Required status check in a ruleset	The same GitHub ruleset, fed by a status the Jenkins GitHub plugin reports
`actions/upload-artifact`	`archiveArtifacts artifacts: '*.tgz'`

The difference that matters is that somebody has to run the Jenkins server. That is a real job, and it is why hosted CI won for small teams.

13. Looking ahead

Your pipeline has one stage that does everything. Lab 07 splits it into named stages with distinct responsibilities, adds static analysis, and makes a **quality gate** fail the build on something other than a test — which raises a question to think about before then: if coverage drops by one per cent, should the build go red?

> **Do not delete anything.** Your workflow, your ruleset and your artifact are all used again. Lab 08 is the mid-term laboratory examination: you build a pipeline from an empty repository under examination conditions, and everything you typed today you will type again without the manual.